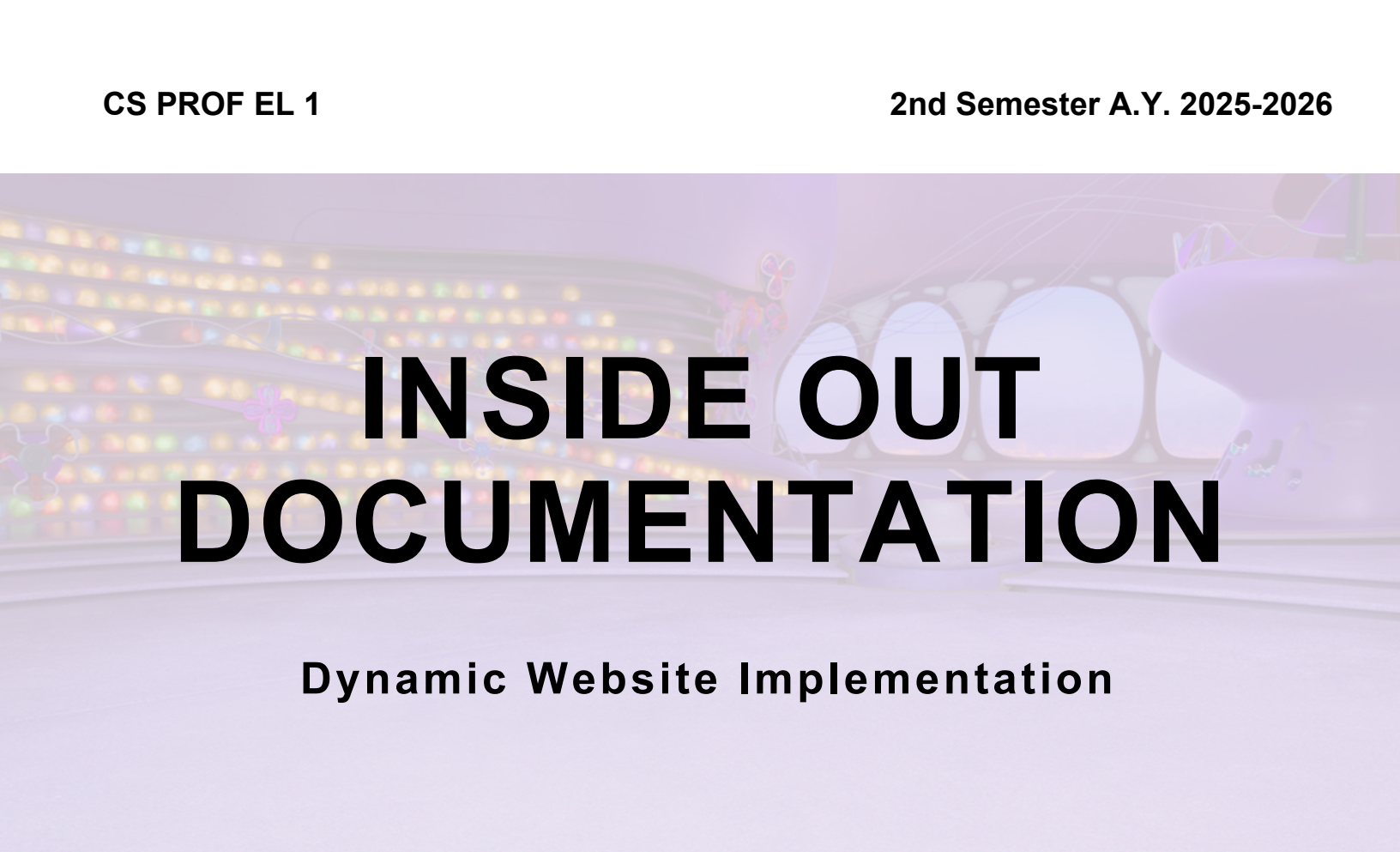




INSIDE OUT DOCUMENTATION

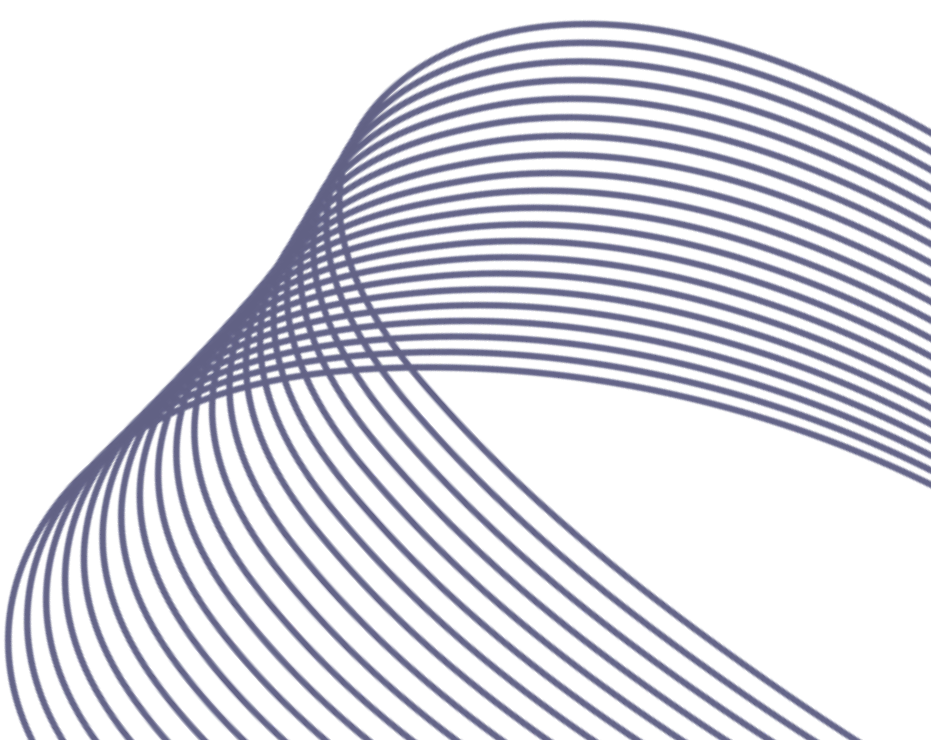
Dynamic Website Implementation

Submitted by:

De Guzman, Hannah Bea A.
BSCS - II

Submitted to:

Prof. Solejaena D. Herrera



System Overview

This website is an informational and interactive system based on Inside Out, an animated film produced by Pixar Animation Studios. It is designed to present the concept of human emotions in a more engaging and easy-to-understand way by allowing users to explore the story, characters, and emotional elements of the movie. The system provides detailed information about each of the core emotions, including their names, images, personality traits, and roles in influencing the main character's thoughts and decisions. Through this, users are able to gain a better understanding of how emotions affect behavior and how they are represented within the film's storyline. The website also includes a watch section where users can view official trailers and access related video content that supports their exploration of the movie.

In addition to its informational features, the system includes a contact module that allows users to send messages, feedback, and suggestions. These inputs are stored in the database and can be reviewed by the administrator, making the system more interactive and user-driven. A login system is also implemented to provide secure access to the admin panel, ensuring that only authorized users can manage and modify the backend content of the system.

The admin page serves as the backend management interface of the website, where the administrator can efficiently handle all content within the platform. This includes creating, reading, updating, and deleting information related to pages, emotions, character descriptions, and video content. The admin dashboard also displays summary data such as the total number of emotions and messages received, allowing the administrator to easily monitor system activity and overall content status.

Overall, the system is developed as a dynamic website, where all content is stored in a database and displayed dynamically using PHP and MySQL. Instead of being hardcoded in static pages, the data is retrieved and managed using SQL queries such as SELECT, INSERT, UPDATE, and DELETE through the admin panel. This allows real-time updates on the user interface without the need to directly modify the source code. As a result, the system becomes easier to manage, more flexible, and scalable for future improvements.

Database Schema

tbl_admin		
Field	Type	Description
admin_id	INT (PK)	Unique ID for each admin
admin_username	VARCHAR	Username used for login
admin_password	VARCHAR	Password for authentication
admin_fname	VARCHAR	First name of admin
admin_mname	VARCHAR	Middle name of admin
admin_lname	VARCHAR	Last name of admin
created_at	DATETIME	Date when account was created

tbl_pages		
Field	Type	Description
page_id	INT (PK)	Unique ID for page
page_name	VARCHAR	Name of the page
created_at	DATETIME	Date created

tbl_content		
Field	Type	Description
content_id	INT (PK)	Unique content ID
page_id	INT (FK)	References tbl_pages
title	VARCHAR	Content title (optional)
description	TEXT	Main content text
image	VARCHAR	Image filename/path
display_order	INT	Order of display
updated_at	DATETIME	Last update

tbl_emotions		
Field	Type	Description
emotion_id	INT (PK)	Unique emotion ID
emotion_name	VARCHAR	Name of emotion
emotion_image	VARCHAR	Image filename/path
display_order	INT	Order of display
updated_at	DATETIME	Last update

tbl_characters		
Field	Type	Description
character_id	INT (PK)	Unique character ID
emotion_id	INT (FK)	References tbl_emotions
character_name	VARCHAR	Name of character
character_role	VARCHAR	Role in story/movie
character_image	VARCHAR	Image filename/path
character_description	TEXT	Character details
updated_at	DATETIME	Last update

tbl_watch		
Field	Type	Description
watch_id	INT (PK)	Unique video ID
video_title	VARCHAR	Title of video
video_link	VARCHAR	Link to video
watchbutton_link	VARCHAR	Button redirect link
display_order	INT	Display order
updated_at	TEXT	Last update

tbl_messages		
Field	Type	Description
message_id	INT (PK)	Unique message ID
message_name	VARCHAR	Sender name
message_email	VARCHAR	Sender email
message_text	TEXT	Message content
created_at	DATETIME	Date sent

Explanation of CRUD Implementation

In this Inside Out website, CRUD operations were used to manage the different contents and modules of the system. CRUD stands for Create, Read, Update, and Delete. These operations are important because they allow the administrator to handle all the data inside the website in an organized way. Through the admin panel, the administrator can add new records, view existing data, edit information when needed, and remove unnecessary data. The system was developed using PHP and MySQL, where PHP is responsible for the backend logic, and MySQL is used to store and manage all the data in the database. This approach makes the system more flexible because any updates in the admin panel will automatically reflect on the website without needing to manually change the source code.

The system follows a modular file structure, where each module has separate PHP files for each CRUD function. For example, in the Emotions module, the files are organized as follows:

- adminemotions.php - displays all emotion records
- emotions_add.php - used for adding new emotions
- edit_emotions.php - used for updating existing emotion data
- deleteemotions.php - used for deleting emotion records

This same structure is also applied to other modules like Content, Characters, and Watch. This makes the system easier to understand, maintain, and debug because each file has a specific responsibility.

Create Operation

The Create operation is used to add new records to the database. In the system, this is implemented through forms where the administrator inputs data and submits it using the POST method. This operation can be seen in files like contentpages_add.php, content_add.php, emotions_add.php, characters_add.php, and watch_add.php.

For example, in emotions_add.php, the administrator can add a new emotion by entering the emotion name, uploading an image, and setting the display order. The form uses:

```
<form method="POST" enctype="multipart/form-data">
```

The enctype="multipart/form-data" is required because the system supports image uploading.

When the form is submitted, the system checks the request method:

```
if ($_SERVER["REQUEST_METHOD"] == "POST")
```

After that, the input values are stored in variables:

```
$emotion_name = $_POST['emotion_name'];  
$display_order = $_POST['display_order'];
```

For image uploads, the system uses the \$_FILES superglobal:

```
$emotion_image = $_FILES['emotion_image']['name'];  
$tmp_name = $_FILES['emotion_image']['tmp_name'];
```

Then the uploaded image is moved to the uploads folder:

```
move_uploaded_file($tmp_name, "uploads/" . $emotion_image);
```

After collecting all the required data, the system inserts it into the database using an SQL INSERT query:

```
INSERT INTO tbl_emotions (emotion_name, emotion_image, display_order, updated_at)  
VALUES ('$emotion_name', '$emotion_image', '$display_order', NOW() )
```

The same Create process is applied in other modules. For example:

- In content_add.php, a new content page is added with title, description, image, and display order.
- In characters_add.php, a character is created and linked to an emotion using emotion_id.
- In watch_add.php, video details such as title, video link, and button link are inserted into tbl_watch.

After a successful insert, the system uses session messages to notify the admin:

```
$_SESSION['status'] = "Successfully Added!";
```

Then it redirects back to the main admin page:

```
header("Location: adminemotions.php");
```

This helps improve user experience because the admin immediately sees confirmation that the data was added.

Read Operation

The Read operation is responsible for displaying data stored in the database. This is mainly used in all admin listing pages such as:

- adminemotions.php
- admincontent.php
- admincharacters.php
- adminwatch.php

This operation uses SQL SELECT queries to retrieve data:

```
SELECT * FROM tbl_emotions
```

The result of the query is stored in a variable:

```
$query = mysqli_query($conn, "SELECT * FROM tbl_emotions");
```

Then the system loops through the results using:

```
while ($row = mysqli_fetch_array($query))
```

Inside the loop, each record is displayed inside an HTML table:

```
<?php echo $row['emotion_name']; ?>
```

For image fields, the system dynamically loads the uploaded image:

```

```


This makes the website dynamic because the content is not static and is pulled directly from the database.

The Read operation is also used in dashboard statistics. For example, the total number of emotions and messages is displayed using:

```
mysqli_num_rows(mysqli_query($conn, "SELECT * FROM tbl_emotions"));
```

This allows the admin dashboard to show real-time data.

Update Operation

The Update operation is used to modify existing records in the database. This is implemented in edit pages such as:

- edit_emotions.php
- edit_content.php
- edit_characters.php
- edit_watch.php

The update process starts by getting the ID of the record:

```
$emotion_id = (int) $_GET['id'];
```

Then the system retrieves the existing data:

```
SELECT * FROM tbl_emotions WHERE emotion_id='$emotion_id'
```

This allows the form to display current values so the admin can edit them easily.

When the form is submitted, the system checks:

```
if ($_SERVER["REQUEST_METHOD"] == "POST")
```

Then updated values are stored:

```
$emotion_name = $_POST['emotion_name'];  
$display_order = $_POST['display_order'];
```

For modules with image uploads, there is a condition to check if a new image is uploaded:

```
if (!empty($_FILES['emotion_image']['name']))
```

If true, the image is updated together with other fields:

```
UPDATE tbl_emotions SET emotion_name='$emotion_name', emotion_image='$new_image',  
display_order='$display_order', updated_at=NOW() WHERE emotion_id='$emotion_id'
```

If no image is uploaded, only text fields are updated.

After updating, the system sets a session message:

```
$_SESSION['status'] = "Successfully Updated!";
```

Then redirects back to the admin page.

This ensures that updates are smooth and do not require re-uploading images unless necessary.

Delete Operation

The Delete operation is used to remove records from the database. This is implemented in files like:

- deleteemotions.php
- deletecontent.php
- deletecharacters.php
- deletewatch.php
- deletecontentpage.php

The system gets the ID of the record using GET:

```
$emotion_id = (int) $_GET['id'];
```

Then it executes a DELETE query:

```
DELETE FROM tbl_emotions WHERE emotion_id='$emotion_id'
```

This removes the selected record permanently from the database.

After deletion, the system sets a session message:

```
$_SESSION['status'] = "Successfully Deleted!";
```

Then it redirects back to the admin page.

To make the system safer, a confirmation prompt is also used in the frontend:

```
onclick="return confirm('Are you sure you want to delete this record?');"
```

This helps prevent accidental deletion of important data.

Overall, the CRUD implementation of the Inside Out website plays a very important role in managing the entire system. It allows the administrator to efficiently control all website data such as pages, content, emotions, characters, and watch videos through a centralized admin panel. Each operation in CRUD is clearly separated into different PHP files, making the system more organized and easier to maintain.

By using PHP and MySQL together, the system becomes dynamic, meaning all changes made in the admin panel are automatically reflected on the website. This reduces the need for manual editing of code and improves efficiency. And through this CRUD system, the Inside Out website becomes more interactive, manageable, and scalable, allowing continuous updates and improvements without affecting the core structure of the website.

Explanation of Search Feature

This feature is mainly implemented in modules such as the Characters and Emotions sections, where the administrator can search for specific records without manually scrolling through all the data. In the Characters module, the admin can search for character names, roles, or descriptions, while in the Emotions module, the admin can search for emotion names or related details. This makes data management faster and more convenient, especially when handling multiple records in the database.

The search function works by using the GET method in PHP, where the input from the search box is passed through the URL. The system then checks if a search value exists and uses it to filter the database results.

For example, in the Characters module, the search input is written as:

```
<input type="text" name="search" class="form-control form-control-sm me-2"
placeholder="Search character..." value="<?php echo isset($_GET['search']) ?
$_GET['search'] : ""; ?>">
```

When the form is submitted, the system checks if a search keyword is provided:

```
if (isset($_GET['search']))
```

The input is then stored in a variable:

```
$search = mysqli_real_escape_string($conn, $_GET['search']);
```

After that, the system uses an SQL query with the LIKE operator to filter results:

```
SELECT * FROM tbl_characters
WHERE character_name LIKE '%$search%'
OR character_role LIKE '%$search%'
OR character_description LIKE '%$search%'
```

For the Emotions module, the same concept is applied, where the system filters records based on the emotion name using a similar LIKE query:

```
SELECT * FROM tbl_emotions
WHERE emotion_name LIKE '%$search%'
```

The LIKE operator allows partial matching, which means that even partial input from the user can still return relevant results. This makes the search feature more flexible and efficient.

If no search input is provided, the system simply displays all records using a standard SELECT query:

```
SELECT * FROM tbl_characters ORDER BY character_id ASC
```

The results are then displayed using a while loop:

```
while ($row = mysqli_fetch_array($query))
```

With that, the search feature improves the usability of the system by allowing the administrator to quickly locate specific records in both the Characters and Emotions modules. It reduces manual effort and makes data management more efficient, especially when handling multiple entries stored in the database.

Explanation of Login System

The website includes a login system that is used to secure the admin panel. This feature ensures that only registered and authorized users can access the backend of the system. The login system is built using PHP for the server-side process and MySQL for checking user credentials stored in the database.

In this system, the login process is handled in the login.php file. The administrator enters their username and password in a form. When the login button is clicked, the system sends the data using the POST method:

```
if (isset($_POST["login"]))
```

The username input is cleaned using `mysqli_real_escape_string()` to prevent invalid or harmful input:

```
$admin_username = mysqli_real_escape_string($conn, $_POST['admin_username']);
```

Instead of storing passwords in plain text, the system uses bcrypt hashing, which provides stronger and more secure password protection compared to basic hashing methods. The password entered by the user is verified using:

```
password_verify($admin_password, $row['admin_password']);
```

After that, the system runs an SQL query to check if the username exists in the admin table:

```
SELECT * FROM $tablelogin  
WHERE admin_username='$admin_username'
```

If a match is found and the password is correct, the system allows the user to log in and creates a session:

```
$_SESSION['admin_username'] = $admin_username;
```

Then the user is redirected to the admin dashboard:

```
header("Location: admindashboard.php");
```

If the login credentials are incorrect, the system displays an error message:

```
$error = "Invalid username and/or password.";
```

The system also includes a registration page found in register.php, where new admin accounts can be created. The user is required to input personal information such as first name, middle name, last name, username, and password.

Before saving the data, the system checks if all required fields are filled and if the password matches the confirm password field:

```
if ($admin_password != $confirm_password)
```

It also checks if the username already exists in the database:

```
SELECT admin_username FROM $tablelogin WHERE admin_username='$admin_username'
```

If the username is available, the password is securely hashed using bcrypt before storing it in the database:

```
$hashed_password = password_hash($admin_password, PASSWORD_DEFAULT);
```

Then the data is inserted into the admin table:

```
INSERT INTO $tablelogin ( admin_username, admin_password, admin_fname, admin_mname,  
                        admin_lname, created_at )
```

After successful registration, the user is redirected to the login page:

```
header("Location: login.php");
```

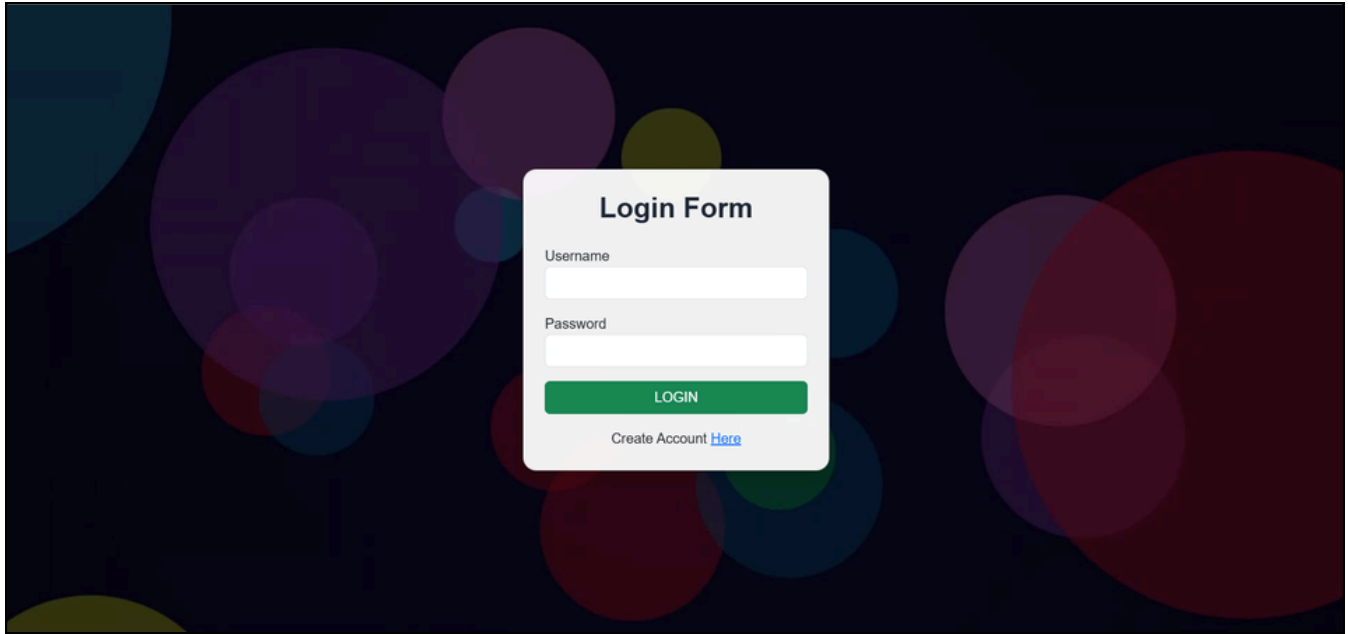
The login system also uses PHP sessions to maintain user authentication. Once the user logs in successfully, the session stores the username:

```
$_SESSION['admin_username']
```

This session is used in different admin pages such as `admindashboard.php`, `admincontent.php`, `admincharacters.php`, and others. It allows the system to display the logged-in admin name and restrict access to unauthorized users.

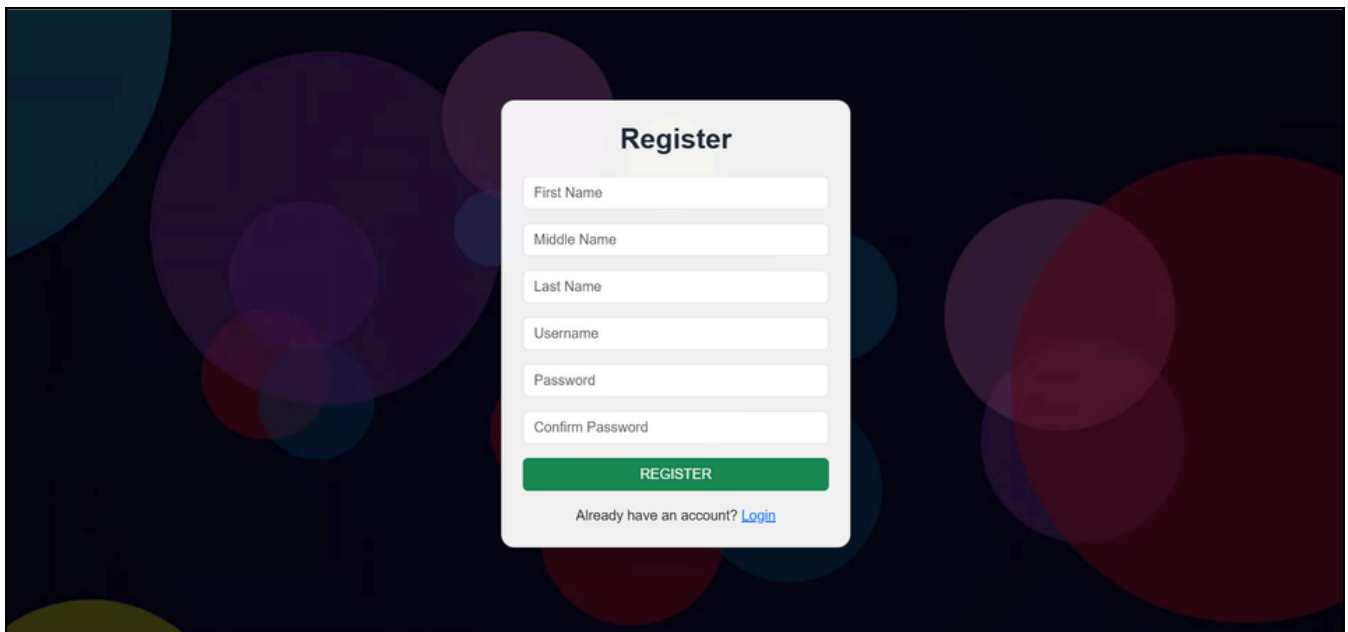
In conclusion, the login and registration system provides security and controlled access to the admin panel. It uses PHP sessions for authentication, `bcrypt` hashing for password security, and MySQL queries for validating user data. This ensures that only registered admins can manage the content of the Inside Out website.

Screenshots of System Features



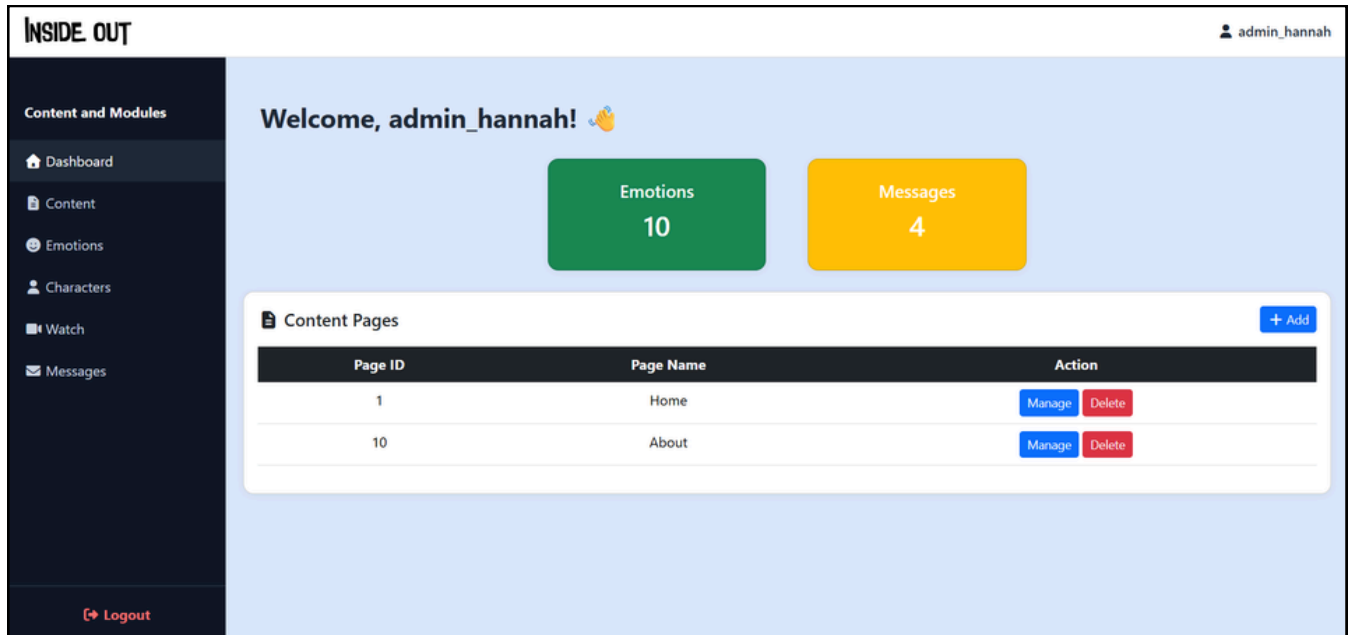
A screenshot of a web application's login form. The form is a white card with rounded corners centered on a dark background with large, overlapping, semi-transparent circles in shades of purple, blue, and red. The card has a title 'Login Form' in bold. Below the title are two input fields: 'Username' and 'Password'. A green button labeled 'LOGIN' is positioned below the password field. At the bottom of the card, there is a link that says 'Create Account [Here](#)'.

Admin Login



A screenshot of a web application's register form. The form is a white card with rounded corners centered on the same dark background with large, overlapping, semi-transparent circles as the login form. The card has a title 'Register' in bold. Below the title are five input fields: 'First Name', 'Middle Name', 'Last Name', 'Username', and 'Password'. A sixth input field labeled 'Confirm Password' is located below the password field. A green button labeled 'REGISTER' is positioned below the confirm password field. At the bottom of the card, there is a link that says 'Already have an account? [Login](#)'.

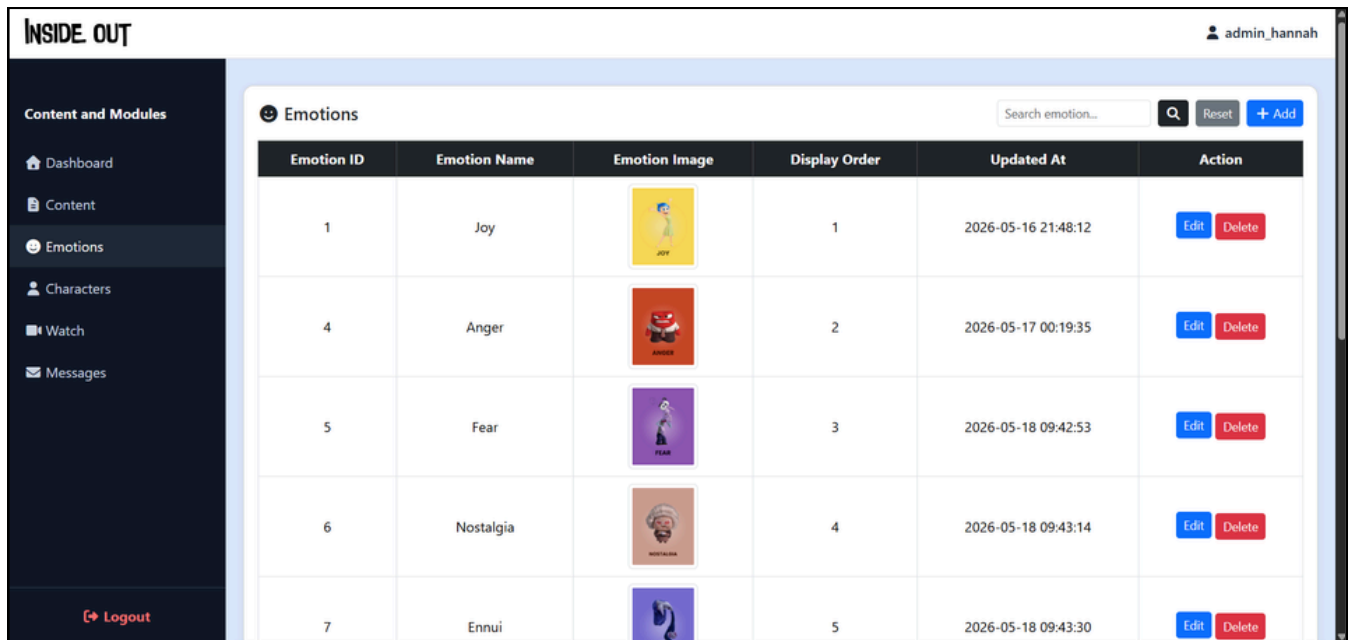
Admin Register



The screenshot shows the 'Admin Dashboard' for 'INSIDE. OUT'. The user 'admin_hannah' is logged in. The dashboard features a sidebar with navigation links: Dashboard, Content, Emotions, Characters, Watch, and Messages. The main content area displays a welcome message and two summary cards: 'Emotions 10' and 'Messages 4'. Below these is a 'Content Pages' table with two rows: 'Home' (Page ID 1) and 'About' (Page ID 10). Each row has 'Manage' and 'Delete' buttons. An '+ Add' button is located at the top right of the table.

Page ID	Page Name	Action
1	Home	Manage Delete
10	About	Manage Delete

Admin Dashboard



The screenshot shows the 'Emotions' section of the 'INSIDE. OUT' admin dashboard. The user 'admin_hannah' is logged in. The section includes a search bar, a 'Reset' button, and an '+ Add' button. Below is a table listing emotions with columns: Emotion ID, Emotion Name, Emotion Image, Display Order, Updated At, and Action. The table contains five rows of data.

Emotion ID	Emotion Name	Emotion Image	Display Order	Updated At	Action
1	Joy		1	2026-05-16 21:48:12	Edit Delete
4	Anger		2	2026-05-17 00:19:35	Edit Delete
5	Fear		3	2026-05-18 09:42:53	Edit Delete
6	Nostalgia		4	2026-05-18 09:43:14	Edit Delete
7	Ennui		5	2026-05-18 09:43:30	Edit Delete

Display Content

INSIDE. OUT

admin_hannah

Content and Modules

Dashboard

Content

Emotions

Characters

Watch

Messages

Logout

Add Emotion

Emotion Name

Emotion Image

Choose File No file chosen

Display Order

ADD CLEAR BACK

Add Content

INSIDE. OUT

admin_hannah

Content and Modules

Dashboard

Content

Emotions

Characters

Watch

Messages

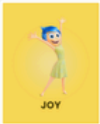
Logout

Edit Emotion

Emotion Name

Joy

Current Image

JOY

Change Image

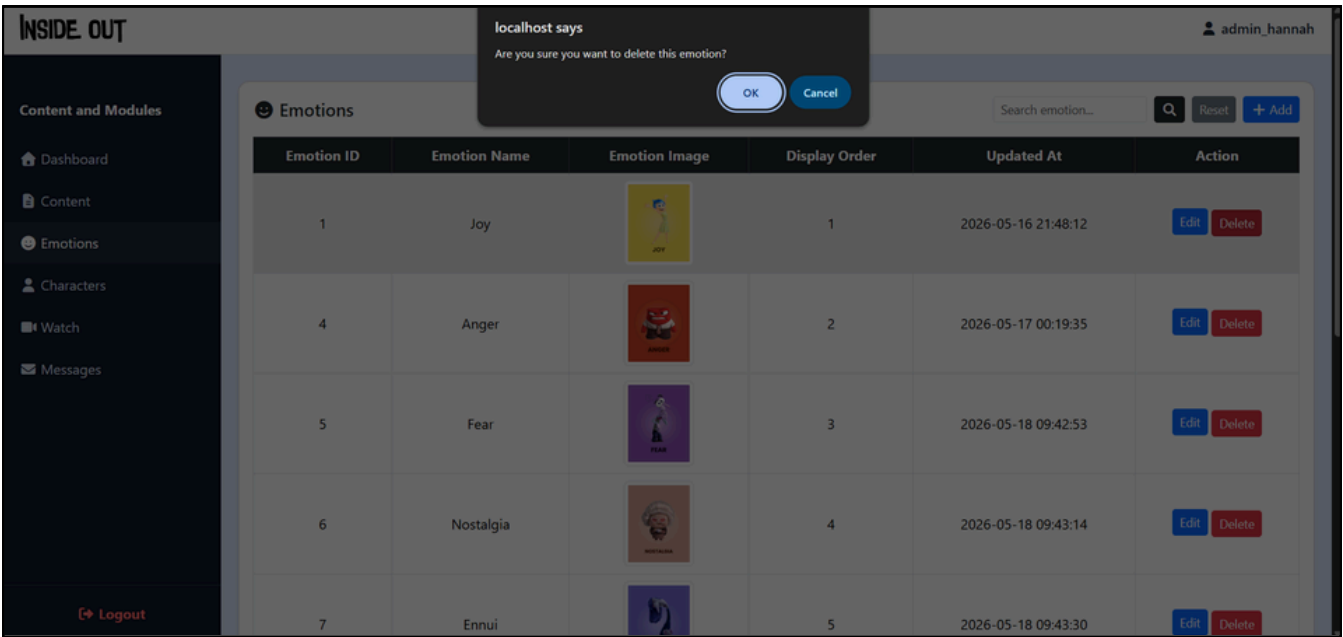
Choose File No file chosen

Display Order

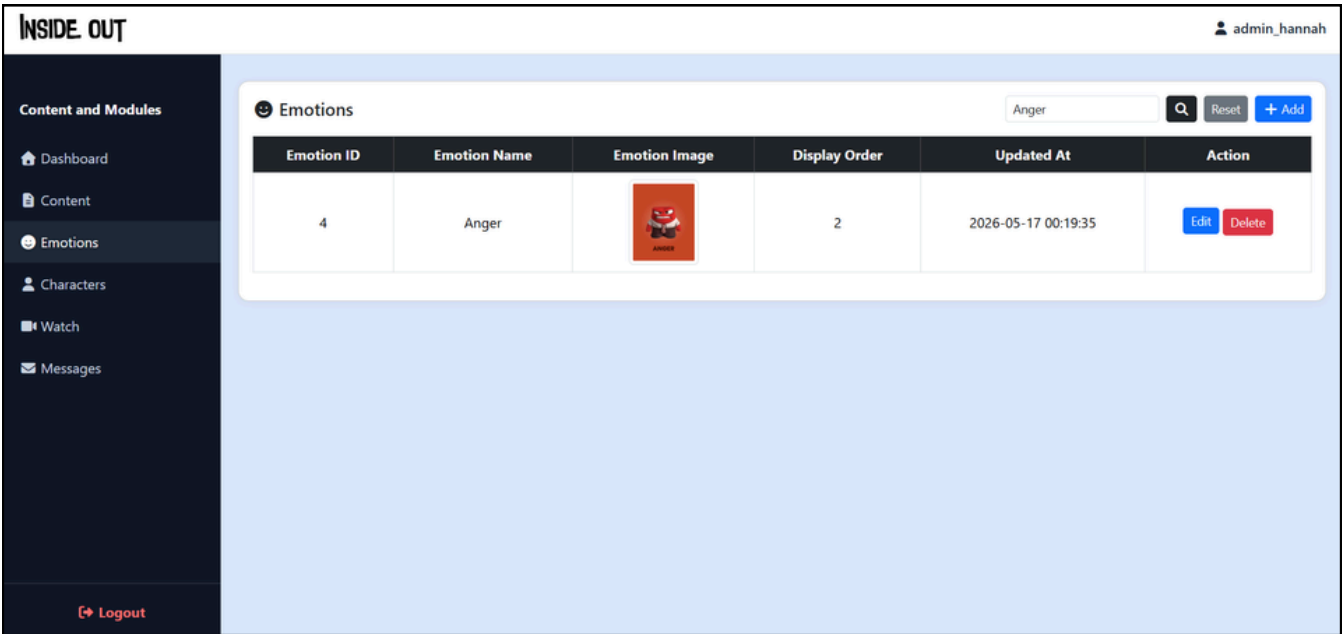
1

UPDATE CLEAR BACK

Edit Content



Delete Content



Search Feature

INSIDE. OUT admin_hannah

Admin Profile

Content and Modules

- Dashboard
- Content
- Emotions
- Characters
- Watch
- Messages

Logout

First Name
Hannah Bea

Middle Name
Adao

Last Name
De Guzman

Username
admin_hannah

New Password

Confirm Password

UPDATE PROFILE CLEAR

Edit Admin Profile

INSIDE. OUT admin_hannah

Content and Modules

- Dashboard
- Content
- Emotions
- Characters
- Watch
- Messages

Logout

Messages

Message ID	Name	Email	Message	Created At	Action
4	Bea De Guzman	bea@gmail.com	hola hola hola	2026-05-20 10:40:49	Delete
3	Jennie Kim	blackpinkjennie@gmail.com	This that pretty girl mantra	2026-05-18 23:55:38	Delete
2	Lalisa Manoban	blackpinklisa@gmail.com	Drop some money, droppin all my money!	2026-05-18 23:52:11	Delete
1	De Guzman, Hannah Bea Adao	hannahbea.adaodeguzman@gmail.com	hi	2026-05-17 00:37:29	Delete

Messages Received From Contact Form